Examination 2025-02-02

Introduction to Parallel Programming (5 hours)

Please make sure to write your exam code on each page you hand in. When you are finished, please put the pages containing your solutions together in an order that corresponds to the order of the questions. You can take with you this exam if you want.
This is a **closed book** examination but you are allowed to have with you an **A4 sheet of prepared hand-written notes**. The exam contains **40** points in total and their distribution between sub-questions is clearly shown. To get a final grade of **5** you must score at least **34**. To get a grade of **4** you must score at least **28** and to pass the exam you must score at least **19**. Note that you will get **credit only for answers that are correct (or a very serious attempt) and clear**. All answers should preferably be **in English**; if you are uncomfortable with English you might of course use Swedish. Whenever in *real doubt* for what a particular question might mean, **state your assumptions clearly**.

DON'T PANIC!

1. **General Questions (2 + 2 + 3 = 7 pts).** Give brief answers to the following questions:
   (a) What is Amdahl's law and what consequences does it have for parallel programming?
   (b) How can we implement a barrier with a condition variable?
   (c) What types of mutexes does Pthreads support? What are the differences between them?

---

2. **Parallel Speedup (3 pts).** A parallel program that obtains a speedup greater than p (the number of processes or threads) is sometimes said to have *superlinear speedup.* However, many authors do not count programs that overcome "resource limitations" as having superlinear speedup. For example, a program that must use secondary storage for its data when it is run on a single processor system might be able to fit all its data into main memory when run on a large multicore or distributed memory system. Give another example on how a program might overcome a resource limitation and obtain speedups greater than p.

---

3. **Safe Locking (5 pts).** The linked list operations `insert` and `delete` consist of two distinct "phases." In the first phase, both operations search the list for either the position of the new node or the position of the node to be deleted. If the outcome of the first phase so indicates, in the second phase a new node is inserted or an existing node is deleted. In fact, it is quite common for linked list programs to split each of these operations into two function calls. For both operations, the first phase involves only read-access to the list; only the second phase modifies the list. Would it be safe to lock the list using a read-lock for the first phase? And then to lock the list using a write-lock for the second phase? Explain your answer.

4. **Locks (7 pts).** In practice, almost all lock acquisitions are uncontended, so the most practical measure of a lock's performance is the number of steps needed for a thread to acquire a lock when no other thread is concurrently trying to acquire the lock. Scientists at Cantaloupe-Melon University have devised the following "wrapper" for an arbitrary lock, shown below.

```
class FastPath implements Lock {
  private static ThreadLocal<Integer> myIndex;
  private Lock lock;
  private int x, y = -1;
  public void lock() {
    int i = myIndex.get();
    x = i;                    // I am here
    while (y != -1) {}        // is the lock free?
    y = i;                    // me again?
    if (x != i)               // Am I still here?
      lock.lock();            // slow path
  }
  public void unlock() {
    y = -1;
    lock.unlock();
  }
}
```

They claim that if the base `Lock` class provides mutual exclusion and is starvation-free, so does the `FastPath` lock, but it can be acquired in a constant number of steps in the absence of contention. Sketch an argument why they are right, or provide a counterexample that shows that they are wrong.

---

5. **Pthread Programming (4 pts).** Imagine a situation with two threads, a global variable named `condition`, a mutex lock, and a function whose code is shown below.

```
int me = ThreadID.get();
do{
  condition = me;
  mutex.lock();
  printf("I am thread %d\n", ThreadID.get());  // prints the thread number
  mutex.unlock();
}while(condition != me);
```

If we initialize thread 1 and thread 2, and upon creation they run this function, what would the outcome (or outcomes) be? Is the result deterministic? Explain your answer and how all the possible outcome(s) can be produced.

---

6. **OpenMP Programming (4 pts).** A student that was passing by the lecture hall of the OpenMP lecture heard that with a simple `#pragma omp parallel for` he can parallelize a serial code in just a minute! He applied the "advice" in the `for` loops of his program, he run it on a (large) multicore but did not notice any time difference. Explain to him possible reasons for the behaviour he is experiencing (the more, the merrier).

7. **More OpenMP Programming (2 + 2 = 4 pts total).** Consider the following program to compute π using Monte Carlo methods. It calculates π based on the number of random points within a unit circle.

```
double x, y, dist, pi;
int i, incircle = 0;

for (i = 0; i < N; i++) {
  x = (double)rand() / RAND_MAX;
  y = (double)rand() / RAND_MAX;

  dist = x * x + y * y;

  if (dist <= 1) {
    ++incircle;
  }
}

pi = 4.0 * incircle / N;
```

Parallelize this program by adding pseudo-OpenMP annotations.

---

8. **MPI Programming (3 + 3 = 6 pts total).** Using a parallel version of the Sieve of Eratosthenes algorithm, a student computes the total number of primes (`total_primes`) in an interval, using `comm_sz` processes, each of which is computing its local count of prime numbers (`local_primes`). She would like to also compute the percentage of total primes that each process produces (i.e., `local_primes/total_primes`), but this requires the number of total primes to be known by all the processes.

   (a) What form of MPI communication do you advise her to use, and why?

   (b) After each process has computed its local `local_primes` and for `comm_sz = 4`, draw a diagram illustrating how communication takes place so that all processes know `total_primes`.

Good luck !